

ASSIGNMENT 1

Coding Environment Setup and Simple API Implementation

He Xiao | UNI: hx2425

Code repository: <https://github.com/HotcatX/API-test>

Part I. API implementation

The existing FastAPI bigram text generator is extended with a word-embedding endpoint. It uses spaCy and the pretrained en_core_web_md 3.8.0 model. For a valid single word, the API returns the model's complete 300-dimensional static vector.

Endpoint	Purpose
GET /embedding?word=apple	Return word, model, dimensions, and embedding (300 floats).
POST /generate	Retain bigram generation using start_word and length.
GET / and GET /docs	Original Hello World response and interactive Swagger documentation.

Run on the instructor's machine

```
git clone https://github.com/HotcatX/API-test.git
cd API-test
docker build -t api-test .
docker run --rm -p 127.0.0.1:8000:80 api-test
```

Open **<http://127.0.0.1:8000/docs>**, expand GET /embedding, select Try it out, enter apple, and Execute. Docker must be installed and running. The image build downloads the locked dependencies and model; requests need no API key or model download.

```
curl 'http://127.0.0.1:8000/embedding?word=apple'
```

Observed result: word=apple, model=en_core_web_md, dimensions=300. First five coordinates, rounded for display: [-0.633400, 0.189810, -0.535440, -0.526580, -0.300010]. The API and assignment/api_example.json contain all 300 coordinates.

Design: app/main.py defines routes and schemas; app/embedding_model.py loads and caches the model and reads Token.vector. Input is trimmed and lowercased. Invalid or out-of-vocabulary words return 422; an unavailable model returns 503. Runtime dependencies are locked in uv.lock; local installation uses the project .venv.

Part II. Rules of Probability

Notation: 'A and B' denotes intersection, 'A or B' denotes union, and 'not A' denotes the complement. Conditional probabilities use $P(A | B)$. The following solutions show the rule, substitution, and result for each question.

Question 1. Independent events

Given $P(A)=0.4$ and $P(B)=0.3$, with A and B independent.

$$\begin{aligned} \text{(a) } P(A \text{ and } B) &= P(A) P(B) \\ &= 0.4 * 0.3 = 0.12 = 3/25. \end{aligned}$$

$$\begin{aligned} \text{(b) } P(A \text{ or } B) &= P(A) + P(B) - P(A \text{ and } B) \\ &= 0.4 + 0.3 - 0.12 = 0.58 = 29/50. \end{aligned}$$

The intersection is subtracted in part (b) so that outcomes in both events are counted only once.

Question 2. Are the events independent?

Given $P(A)=0.5$, $P(B)=0.4$, and $P(A | B)=0.7$. Because $P(B)>0$, independence requires $P(A | B)=P(A)$. Here, 0.7 differs from 0.5, so **A and B are not independent**. Knowing B occurred changes the probability of A.

$$\begin{aligned} P(A \text{ and } B) &= P(A | B) P(B) = 0.7 * 0.4 = 0.28. \\ P(A) P(B) &= 0.5 * 0.4 = 0.20. \\ \text{Since } 0.28 &\neq 0.20, \text{ the product rule also fails.} \end{aligned}$$

Question 3. Bayes' rule

Given $P(A)=0.6$, $P(B | A)=0.5$, and $P(B | \text{not } A)=0.2$. First use the law of total probability, then Bayes' rule.

$$\begin{aligned} P(\text{not } A) &= 1 - 0.6 = 0.4. \\ P(B) &= P(B | A) P(A) + P(B | \text{not } A) P(\text{not } A) \\ &= 0.5 * 0.6 + 0.2 * 0.4 \\ &= 0.30 + 0.08 = 0.38. \\ P(A | B) &= P(B | A) P(A) / P(B) \\ &= 0.30 / 0.38 = 15/19 \\ &= 0.789473684... \text{ (approximately 78.95\%).} \end{aligned}$$

Question 4. Probability of disease after a positive test

Let D mean disease and T+ mean a positive result. The given rates are $P(D)=0.02$, $P(T+ | D)=0.95$, and $P(T- | \text{not } D)=0.90$. Therefore $P(\text{not } D)=0.98$, and the false-positive rate is $1-0.90=0.10$.

$$\begin{aligned} P(T+) &= P(T+ | D) P(D) + P(T+ | \text{not } D) P(\text{not } D) \\ &= 0.95 * 0.02 + 0.10 * 0.98 \\ &= 0.019 + 0.098 = 0.117. \end{aligned}$$

$$\begin{aligned} P(D | T+) &= P(T+ | D) P(D) / P(T+) \\ &= 0.019 / 0.117 = 19/117 \\ &= 0.162393162... \text{ (approximately 16.24\%).} \end{aligned}$$

Using the supplied rates in a hypothetical group of 10,000: 200 have the disease, of whom 190 test positive; among 9,800 healthy people, 980 test positive. Thus $190/(190+980)=19/117$. Low prevalence explains the difference between this posterior and sensitivity.

Question 5. Expectation, variance, and sample mean

x	P(X=x)	x P(X=x)	x ² P(X=x)
85	0.375	31.875	2709.375
90	0.375	33.750	3037.500
95	0.125	11.875	1128.125
100	0.125	12.500	1250.000
Total	1	90	8125

$$\begin{aligned} \text{(a) } E[X] &= \sum x P(X=x) \\ &= 31.875 + 33.750 + 11.875 + 12.500 = 90. \end{aligned}$$

$$\begin{aligned} \text{(b) } E[X^2] &= 2709.375 + 3037.500 + 1128.125 \\ &\quad + 1250.000 = 8125. \\ \text{Var}(X) &= E[X^2] - (E[X])^2 \\ &= 8125 - 90^2 = 25 \text{ points squared.} \end{aligned}$$

$$\begin{aligned} \text{(c) Sample mean} &= (85+90+85+95+90+85+100+90)/8 \\ &= 720/8 = 90 \text{ points.} \end{aligned}$$

The requested variance is the variance of the given distribution. The sample mean equals $E[X]$ here because the sample proportions (3/8, 3/8, 1/8, 1/8) match the given probabilities. Other random samples need not have mean 90.

Question 6. Entropy in bits

(a) For message probabilities 0.4, 0.3, 0.2, and 0.1, use base-2 logarithms:

$$\begin{aligned} H(X) &= -\sum p(x) \log_2 p(x) \\ &= -[0.4 \log_2(0.4) + 0.3 \log_2(0.3) \\ &\quad + 0.2 \log_2(0.2) + 0.1 \log_2(0.1)]. \end{aligned}$$

Message	p	log ₂ (p)	-p log ₂ (p)
A	0.4	-1.321928095	0.528771238
B	0.3	-1.736965594	0.521089678
C	0.2	-2.321928095	0.464385619
D	0.1	-3.321928095	0.332192809

Adding the contributions without intermediate rounding gives **H(X)=1.8464393446710154 bits**, or approximately **1.84644 bits**.

(b) For four equally likely messages, each probability is 1/4, and log₂(1/4)=-2.

$$\begin{aligned} H(\text{uniform}) &= -4[(1/4) \log_2(1/4)] \\ &= -4[(1/4)(-2)] = 2 \text{ bits.} \end{aligned}$$

This equals log₂(4), the maximum entropy for four possible messages, and exceeds the nonuniform distribution's entropy because the messages are equally unpredictable.

Reproducibility and verification

Local verification: **51 tests passed**; HTTP checks passed for the embedding endpoint, invalid/OOV input, original text generation, and Swagger documentation. The returned vectors were compared with an independently loaded spaCy model.

Environment: Python 3.12.2, spaCy 3.8.16, en_core_web_md 3.8.0.

Docker verification passed on GitHub Actions (Linux): the image was built, the container started, and real HTTP embedding and regression checks passed.

[View the verified workflow run.](#)

```
uv run pytest -q
uv run python scripts/check_probability.py
uv run python scripts/smoke_api.py # server must be running
```

The probability checker uses exact Fraction arithmetic for Questions 1-5 and math.log2 for Question 6. It checks the saved numerical answers against independent calculations.

Sources: Assignment1-1.pdf, Questions 1-6; spaCy vectors documentation; en_core_web_md model documentation.